

CHAPTER 4: DEVELOPMENT AND IMPLEMENTATION

4.1 DEVELOPMENT ENVIRONMENT

The IoT-Based Bus Stop System for Public Transport Overcrowding Prediction Using Machine Learning was developed using embedded hardware and a local Python backend. Arduino IDE with the ESP32 Board Manager was used to write, compile and upload C++ firmware to the ESP32 DoIT DevKit V1. Visual Studio Code was used to develop brain.py in Python 3 on a Windows 11 computer.

Git provided local version control. The Arduino Serial Monitor supported inspection of VL53L0X distance readings, DHT22 temperature values and connection messages. The Blynk IoT Console and mobile application provided the operator interface; they were configured using widgets and virtual pins rather than a separately programmed React or Android application. The backend used scikit-learn for regression and requests for external API communication, while recording each processing cycle in bus_stop_log.csv.

Development tools used: Arduino IDE, ESP32 Arduino Core, Visual Studio Code, Git, Windows 11, Arduino Serial Monitor, Blynk IoT Console and Python 3. The principal Python packages were scikit-learn and requests; csv, time and datetime supported logging and timing.



Figure 4.1: Development environment and principal tools

4.2 IMPLEMENTATION APPROACH

A hardware-first, bottom-up and iterative approach was adopted. Sensor power and communication were checked before cloud and machine-learning integration because incorrect distance readings would propagate into passenger counts and risk estimates. Each module was assembled, exercised and refined before it was incorporated into the complete prototype.

The six phases in Figure 4.2 cover sensor interfacing, ESP32 firmware, backend model development, cloud and logging integration, dashboard configuration, and system integration.

Module checks were followed by an end-to-end demonstration from a confirmed crossing to the dashboard, physical indicators and conditional Telegram alert.



Figure 4.2: Phased implementation from local sensing to connected operation

Implementation proceeded from local measurements to connected operation. The VL53L0X was checked on I2C, the DHT22 on its digital interface, and the 400 mm detection threshold was exercised using an object. Firmware then added two-reading confirmation, cloud updates and operator control. The backend combined these measurements with local time and weather before predicting risk.

Cloud integration established a single writer for each data stream, CSV persistence and alert delivery. Dashboard development bound the V1-V8 widgets to these streams. The final integration stage checked whether sensing, prediction, manual subtraction, LED updates and

alert cooldown behaved consistently. No field-accuracy percentage or latency benchmark is inferred from this development sequence.

4.3 HARDWARE ASSEMBLY

The hardware was assembled on a breadboard to make the power, I2C and GPIO connections accessible during development. The sequence below describes the construction of the bus-stop prototype.

Step 1: Preparation of Hardware Modules

The ESP32 DoIT DevKit V1 was mounted across the breadboard centre trench with its USB connector accessible. A 5 V USB supply powered the development board. Only the board's regulated 3V3 output and GND were routed to the peripheral rails; no sensor was connected to the 5 V/VIN supply.

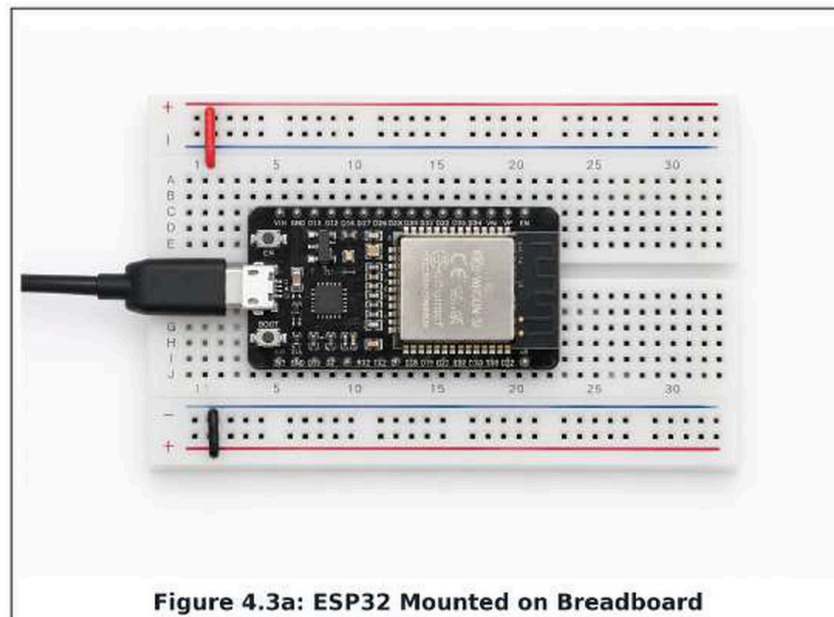


Figure 4.3a: ESP32 mounted on the breadboard

Step 2: VL53L0X Sensor Connection

The VL53L0X Time-of-Flight module was connected using jumper wires: SDA to GPIO21, SCL to GPIO22, supply to 3V3 and ground to common GND. Its sensing aperture was kept unobstructed. Serial Monitor readings were used to check communication and the response to objects inside and outside the 400 mm detection region.

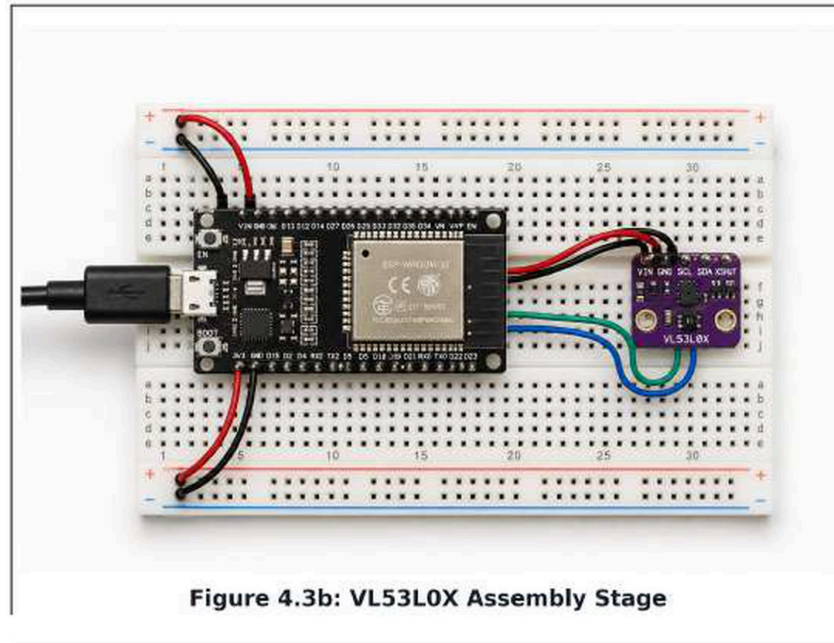


Figure 4.3b: VL53L0X I2C connection

Step 3: DHT22, LEDs and Operator Push Button

The DHT22 data connection was made to GPIO4 and its supply to 3V3. The red LED used GPIO25 and the green LED used GPIO26, with a separate 220-ohm series resistor in each LED branch and a shared ground. The local momentary button connected GPIO27 to GND; the corresponding operator action was also available remotely through Blynk V7. Both inputs invoked the same count-reduction operation.

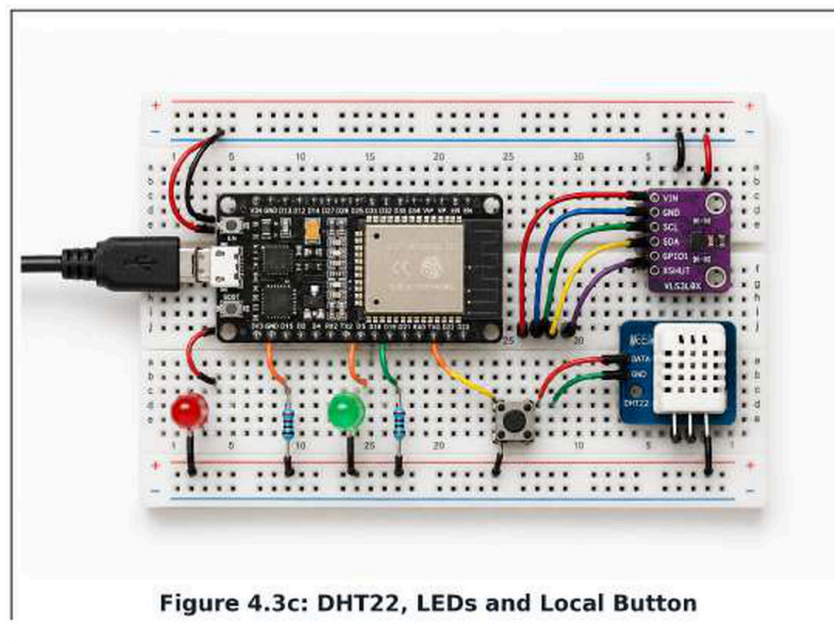


Figure 4.3c: DHT22, LEDs and local button wiring

Step 4: Power Connection and Testing

After checking rail polarity and jumper continuity, USB power was applied and sensor values were inspected in the Serial Monitor. The complete prototype included a buzzer provision driven through a BC547 NPN transistor; audible operation remained disabled in firmware. The assembly remained a breadboard prototype, with enclosure, strain relief and secure sensor mounting required before unattended outdoor use.

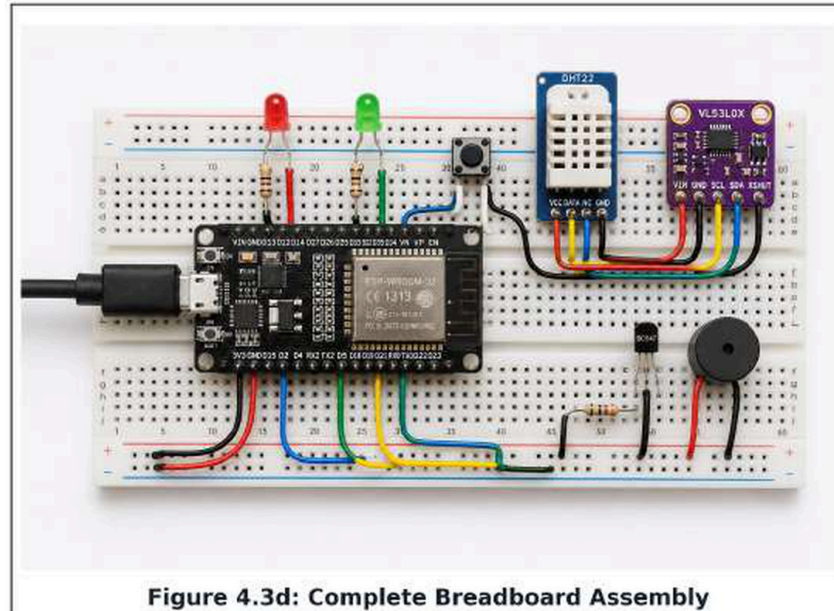


Figure 4.3d: Power and complete breadboard assembly

Tools and materials used: ESP32 DoIT DevKit V1, breadboard, VL53L0X, DHT22 module, red and green 5 mm LEDs, two 220-ohm resistors, momentary push button, jumper wires and USB cable. The disabled buzzer/BC547 provision was retained from the hardware design.

Hardware Connection Record

Component / signal	Connection	Implementation note
USB power	ESP32 USB input	5 V input; on-board regulation
Peripheral supply	ESP32 3V3 and GND	3.3 V peripherals; common ground
VL53L0X SDA / SCL	GPIO21 / GPIO22	I2C distance measurements
DHT22 DATA	GPIO4	Temperature read every 3 s
Red LED	GPIO25	220-ohm series resistor; CRITICAL
Green LED	GPIO26	220-ohm series resistor; NORMAL
Local button	GPIO27 to GND	Input pull-up; debounced press event
Remote operator button	Blynk V7	Virtual input, not a physical GPIO

Buzzer provision	BC547 driver	Present but firmware-disabled; no new GPIO assigned here
------------------	--------------	---

Issues Faced and Fixes

Issue encountered	Solution implemented
Sensors remained unpowered when the breadboard rail was connected to VN, an input-only pin, although the ESP32 was powered by USB.	Moved the rail connection to the actual 3V3 output and retained common GND.
Single-frame distance drops produced ghost counts during breadboard testing.	Required two consecutive readings below 400 mm and a latch that prevents repeated counts until the sensing path clears.
Occasional invalid DHT22 values could enter the data stream.	Discarded NaN readings instead of uploading them as a valid temperature.
ESP32 and Python writes could overwrite shared Blynk values.	Assigned V1/V4/V8 to the ESP32 and V2/V3/V5/V6 to brain.py; V7 remained an operator input.

The local GPIO27 button and the Blynk V7 widget are different interfaces to the same operation, not electrically connected pins. A local button press must be debounced. If an interrupt is used, it should only flag the event; count changes and Blynk communication are handled outside the interrupt routine. Module pull-up requirements must be checked against the fitted DHT22 breakout before assembly.

4.4 FIRMWARE DEVELOPMENT

The firmware was developed in C++ using the ESP32 Arduino Core. It acquires sensor data, confirms crossing events, maintains the passenger count and exchanges values with Blynk. The following focused implementation excerpts express the specified control logic; they are not a complete flash-ready sketch or a byte-for-byte copy of an uploaded source file.

1. Sensor Initialization and Scheduling

```
Wire.begin(21, 22);
if (!I2C.begin()) {
    Serial.println("Failed to boot VL53L0X");
    while (true) { delay(100); }
}
dht.begin();
pinMode(25, OUTPUT);
pinMode(26, OUTPUT);
pinMode(27, INPUT_PULLUP);
Blynk.begin(auth, ssid, pass);
timer.setInterval(150L, checkCrossing);
timer.setInterval(3000L, readTemperature);
```

Wire.begin(21, 22) selects the intended I2C pins. lox.begin() verifies that the distance module responds; dht.begin() prepares the temperature sensor. Timed callbacks check the VL53L0X every 150 ms and read the DHT22 every 3 seconds. Blynk.run() and timer.run() are serviced from the main loop.

2. Crossing Detection and Edge Processing

```
// Executed every 150 ms after a usable range sample.
if (distance_mm < 400) {
    consecutiveDetections++;
    if (consecutiveDetections >= 2 && !isCrossing) {
        passengerCount++;
        lastCrossingMs = millis();
        isCrossing = true;
        Blynk.virtualWrite(V1, passengerCount);
    }
} else {
    consecutiveDetections = 0;
    isCrossing = false;
}
```

The counter increases only after two consecutive readings below 400 mm. The crossing latch then blocks further increments while the object remains in the detection zone. A clear reading re-arms detection. Invalid/out-of-range samples must not be accepted as a below-threshold crossing. This is threshold confirmation and debouncing, not identification of an individual passenger or direction of travel.

3. Temperature and Time-Since-Crossing Updates

```
void readTemperature() {
    float t = dht.readTemperature();
    if (!isnan(t)) Blynk.virtualWrite(V4, t);
}
// Publish periodically after a confirmed crossing exists:
float minutesSince = (millis() - lastCrossingMs) / 60000.0f;
Blynk.virtualWrite(V8, minutesSince);
```

Invalid DHT22 readings are discarded. V8 reports elapsed minutes from the latest confirmed crossing and provides context for operator review; it does not automatically remove passengers or prove that a bus has departed. Before the first confirmed crossing, a displayed zero must be interpreted as an initialized value rather than evidence of a recent event.

4. Operator Action and Physical Indication

```
void applyBusLeft() {
    passengerCount = max(0, passengerCount - 30);
    Blynk.virtualWrite(V1, passengerCount);
}
BLYNK_WRITE(V7) {
    if (param.asInt() == 1) applyBusLeft();
}
// A debounced GPIO27 press calls applyBusLeft() as well.
BLYNK_WRITE(V3) {
    float score = param.asFloat();
    digitalWrite(25, score > 75.0f ? HIGH : LOW);
}
```

```
digitalWrite(26, score <= 40.0f ? HIGH : LOW);
}
```

The V7 widget is configured as a momentary push control. A press subtracts 30 without allowing a negative count. Python publishes the risk value on V3; the ESP32 consumes it and drives the LEDs. CRITICAL lights red, NORMAL lights green, and WARNING is shown as amber on the dashboard with neither of these two physical LEDs asserted. The buzzer is not activated.

5. Wi-Fi and Blynk Connection Management

The main loop regularly services Blynk and the sensor timer. Reconnection requires Wi-Fi/Blynk connection handling; calling Blynk.run() alone is not a guarantee of uninterrupted recovery. After reconnection, the design should republish ESP32-owned values and synchronize V3. Stale-data behaviour and restart recovery must be checked on the deployed firmware rather than assumed from these excerpts.

Libraries Used

Library	Purpose
WiFi.h	ESP32 Wi-Fi connectivity
Wire.h	I2C communication on GPIO21/22
Adafruit_VL53L0X.h	VL53L0X range measurements
DHT.h	DHT22 temperature acquisition
BlynkSimpleEsp32.h	Blynk virtual-pin communication over TCP/IP; this header alone does not establish TLS
BlynkTimer	Non-blocking scheduling of periodic callbacks

4.5 DATABASE / CLOUD INTEGRATION

Blynk Cloud acts as the data bridge between the ESP32, the local Python backend and the operator dashboard. brain.py uses the Blynk HTTPS REST API to retrieve V1 and V4, requests current weather for Dindoshi from OpenWeatherMap, evaluates risk and updates V2, V3, V5 and V6. The intended processing interval is 5 seconds.

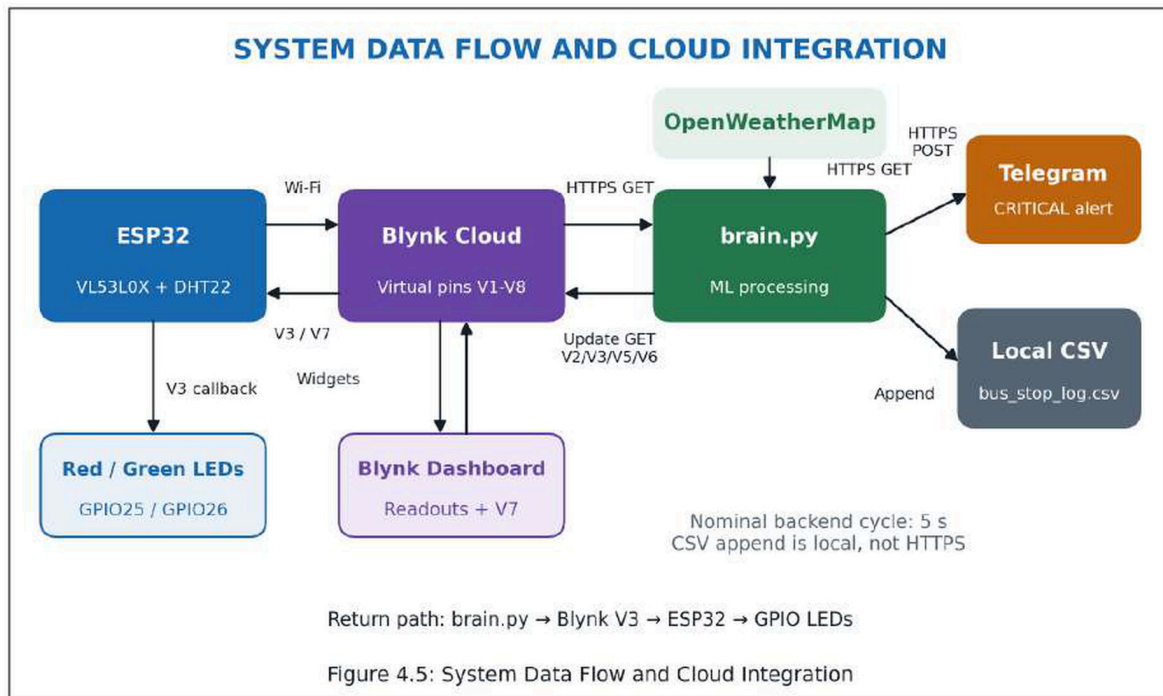


Figure 4.5: Cloud, weather, logging and Telegram update path

The reverse arrows in Figure 4.5 show the return path for risk and status updates; this path is necessary for dashboard and LED control. Telegram receives an HTTPS POST only for an eligible CRITICAL alert. CSV persistence is a local append operation, not an HTTP request. The prototype does not implement the future relational database proposed in Chapter 3.

If V4 is unavailable, the backend substitutes 28.0 degrees Celsius. If weather retrieval fails, it uses Clear and `is_raining = 0`. These fallbacks keep processing available but can underestimate risk; a missing value must not be described as a measured clear-weather or temperature observation.

Virtual-Pin Ownership

Pin	Data / action	Writer / origin
V1	Passenger count	ESP32
V2	Weather text	brain.py
V3	Risk score; ESP32 LED control input	brain.py
V4	Temperature (degrees Celsius)	ESP32
V5	Local date/time	brain.py
V6	Status / routing-action text	brain.py
V7	Bus Left (-30) momentary command	Operator widget
V8	Minutes since last confirmed crossing	ESP32

Cloud Update and CSV Logging Excerpt

```
response = requests.get(
    "https://blynk.cloud/external/api/update",
```

```

params={"token": BLYNK_TOKEN, "v3": risk_score},
timeout=10
)
response.raise_for_status()
with open("bus_stop_log.csv", "a", newline="") as log_file:
    csv.writer(log_file).writerow([
        timestamp, passenger_count, temp, weather,
        risk_score, status
    ])

```

The complete cycle writes all Python-owned output pins, not only V3 shown in the excerpt. The CSV header is timestamp, passengers, temp, weather, risk, status. API calls require exception handling so a failed update or alert does not silently prevent other cycle work. API tokens are supplied from private configuration and excluded from public code and logs.

Configuration	Implementation
Cloud / weather / alerts	Blynk Cloud / OpenWeatherMap / Telegram Bot API
Persistent storage	Local bus_stop_log.csv; no database server
Backend communication	HTTPS GET for Blynk/weather; HTTPS POST for Telegram
Device communication	Blynk protocol over Wi-Fi/TCP/IP; TLS not verified
Authentication	Blynk token, OpenWeatherMap key, Telegram bot token
Data representation	API response values/JSON; local CSV rows
Cycle scheduling	Nominal 5 s; elapsed time also depends on API latency

A `time.sleep(5)` at the end of a loop produces a period of processing time plus 5 seconds, not an exact five-second schedule. Polling at this frequency also needs to respect each service's account quota; it does not by itself prove that rate limits have been eliminated.

4.6 DASHBOARD DEVELOPMENT

The operator dashboard was configured in Blynk IoT Web and Mobile using gauges, value displays, a momentary button and a SuperChart. Its header identifies Dindoshi Bus Depot, BEST Route 326 and stop identifier BEST-DIN-001. Widgets are bound to V1-V8 so the same data ownership established during cloud integration is preserved in the interface.

Operator View

The passenger gauge uses a 0-45 display range, with green for 0-30, amber for 31-40 and red for 41-45. The risk display has a 98-point cap, with NORMAL at 40 or below, WARNING above 40 through 75, and CRITICAL above 75. Weather, temperature, local date/time and the

status/routing message are shown in separate tiles. The risk value is an index displayed with a percent sign, not a calibrated probability.

Manual Control and Historical Display

The blue Bus Left (-30) button is configured on V7 in push mode. The ESP32 applies $\max(0, \text{passenger_count} - 30)$ and republishes V1. V8 shows minutes since the last confirmed crossing. The trend chart uses separately labelled count and risk scales so different units are not confused. Trend points in the supplied mockups are synthetic and end at the displayed values.

Component	Technology used
Web/mobile dashboard	Blynk IoT Console and Blynk App
Value displays	Gauge widgets and value tiles
Historical chart	Blynk SuperChart
Operator control	V7 momentary push button
Data bindings	Virtual pins V1-V8
Critical notification	Telegram Bot API

Interpretation of the Example Screen

Figure 4.6 preserves the requested display example: 38 passengers, 82% risk, Rain, 34 degrees Celsius, 2026-08-26 18:32, CRITICAL and V8 = 4 minutes. The classification is correct because $82 > 75$. However, the reference formula for 38 passengers in rain at 34 degrees Celsius during rush hour gives $\min(98, 63.33 + 15 + 10 + 10) = 98$, not 82. Therefore the screen is explicitly labelled an illustrative UI state, not a recorded model prediction or validation result.

Dashboard verification checks each pin binding, gauge scale, state colour, button action and trend endpoint. Actual Blynk screenshots must be used where evidence of live operation is required; the generated mockups in this chapter do not establish measured performance.

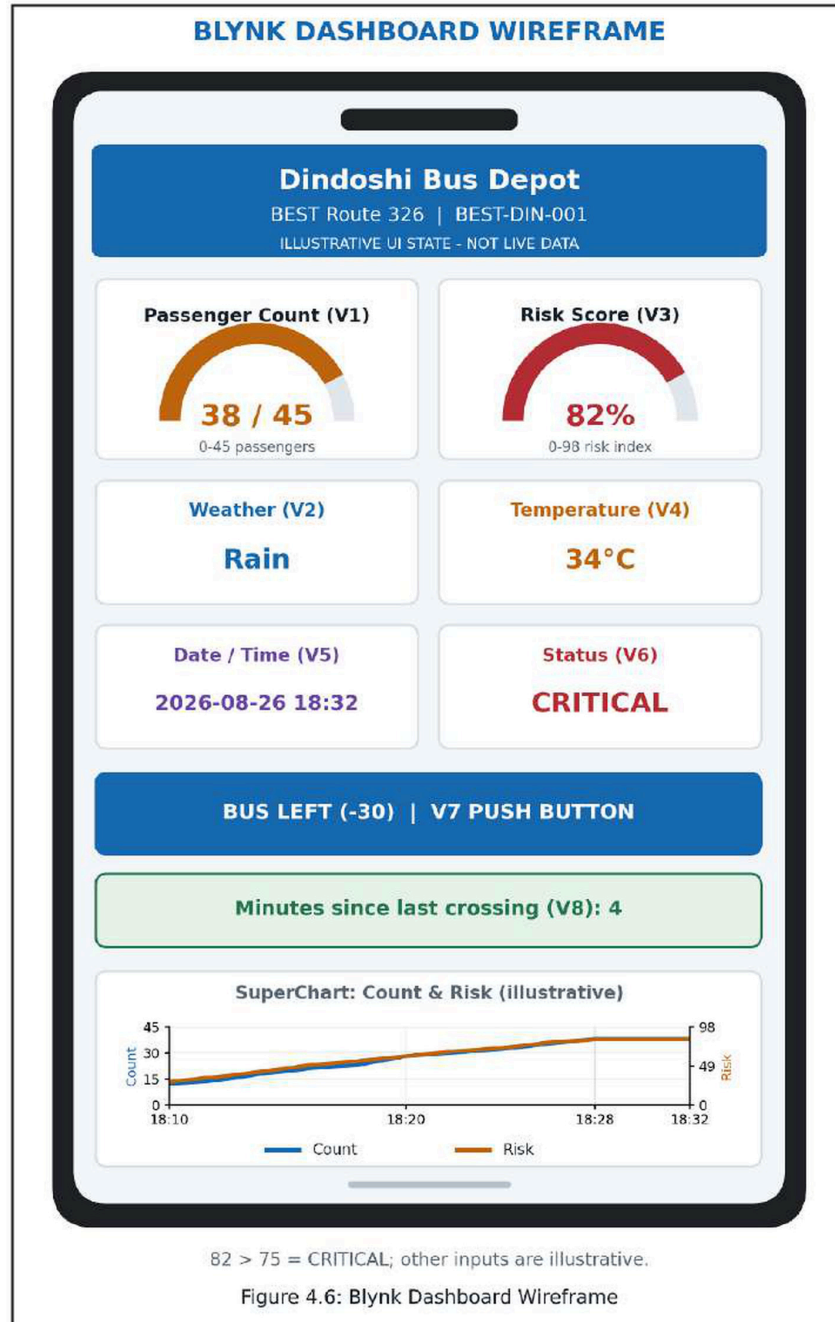


Figure 4.6: Illustrative dashboard state (38 passengers, 82% risk)

4.7 MODULE-WISE IMPLEMENTATION

The module record below follows the implementation, input, output, testing and difficulty structure of the reference chapter. Qualitative outcomes are taken from the supplied project description; no raw test log, error rate or delivery-time measurement was supplied for independent verification.

Module	What was done	Input	Output	Test result*	Difficulty / resolution
VL53L0X	I2C on GPIO21/22; 150 ms sampling; two-hit confirmation	Object distance	Confirmed crossing; V1 count	Below-400 mm blockage detection reported	Single-frame ghost counts; two readings plus re-arm latch
DHT22	GPIO4 acquisition every 3 s; reject NaN	Ambient air	Temperature on V4	Stable readings after initialization reported	Invalid samples; discard instead of publishing
ESP32 / Blynk	Publish V1/V4/V8; consume V3 and V7	Sensors, risk, operator input	Cloud values and GPIO outputs	Sensor-to-cloud operation reported	Conflicting writes; explicit pin ownership
Python ML	RandomForestRegressor: 100 trees, random_state=42	Hour, rain, count, temperature	Risk index and status	Rule-approximation behaviour described; accuracy not quantified	API quota/latency; nominal 5 s pacing, quota review required
Alerts / UI	V3-driven LEDs, dashboard status, Telegram POST	Predicted risk	Red/green output; alert message	Critical alert delivery reported; latency not measured	Repeated alerts; 60 s cooldown
Operator control	Remote V7 and local GPIO27 use one subtraction action	Momentary press	max(0, count - 30)	Non-negative reduction is the acceptance criterion	Duplicate local events; debounce and process outside ISR
CSV logger	Append six fields per completed cycle	Cycle values	bus_stop_log.csv	Logging implementation described; raw file not supplied	Write/API failure must be visible; guard cycle stages

Module Testing Summary

Independent module checks isolate power faults, invalid sensor readings, duplicate detections and cloud-write conflicts. Integration then checks the full path: object detection, V1 update, backend feature construction, V3 prediction, LED response, dashboard status and conditional Telegram delivery. Operator tests include counts below 30 to verify that subtraction stops at zero. The button is a manual estimate of boarding passengers, not automatic bus or occupancy detection.

4.8 ALGORITHMS AND CONTROL LOGIC

The edge algorithm uses a distance threshold with consecutive-sample confirmation and a crossing latch. The backend uses RandomForestRegressor to estimate a risk index from the ordered feature vector [hour, is_raining, passenger_count, temperature]. The specified model uses 100 trees and random_state = 42. Synthetic training labels are derived from the project's reference formula; this demonstrates rule approximation, not validation of real-world crowding probability.

```
Risk_target = min(98, (passenger_count / 45 × 75)
                  + rain_bonus + heat_bonus + rush_hour_bonus)
```

Term	Definition
passenger_count	Live value read from V1
45-person reference	Project scaling assumption; not a certified shelter capacity
rain_bonus	15 when is_raining = 1; otherwise 0
heat_bonus	10 at temperature >= 33 degrees Celsius; otherwise 0
rush_hour_bonus	10 for configured morning/evening rush hours (8-9 AM and 6-7 PM); otherwise 0
Upper bound	98-point target cap

The Random Forest prediction is used as risk_score, while the formula defines its training target. A prediction may differ from that target and should not be presented as an exact formula evaluation without checking the saved model. Neither the training-grid size nor a held-out error metric is invented here.

Condition	Status	Response
risk_score <= 40	NORMAL	Green LED on; red off; no Telegram
40 < risk_score <= 75	WARNING	Amber dashboard state; red/green off; no Telegram
risk_score > 75	CRITICAL	Red LED on; green off; Telegram subject to cooldown

The middle interval includes decimal values such as 40.5. The shorthand 41-75 is valid only for integer display values. Exactly 75.0 is WARNING. The 60-second cooldown applies only to Telegram messages; the risk classification and physical LED state are updated each cycle. A successful send updates the last-alert timestamp so repeated critical cycles do not flood the recipient.

The design uses local Mumbai time for the hour feature and dashboard timestamp. A single object crossing does not automatically produce CRITICAL: the decision depends on accumulated count, temperature, weather and time. Simultaneous crossings and changes in the sensor's physical alignment remain practical limitations of a single-sensor estimate.

Control-Logic Interpretation

PSEUDOCODE: RISK CLASSIFICATION AND ALERTING

```

01 START nominal 5-second brain.py cycle
02 Read passenger_count (V1) and temperature (V4)
03 If V4 unavailable: temperature = 28.0
04 Fetch OpenWeatherMap; on failure use Clear
05 Build [hour, is_raining, passenger_count, temperature]
06 risk_score = RandomForestRegressor.predict(features)
07 IF risk_score > 75 THEN
08     status = "CRITICAL"; red = ON; green = OFF
09     IF no previous alert OR elapsed >= 60 seconds THEN
10         Send Telegram notification via HTTPS POST
11         If successful: update last_alert_time
12     END IF
13 ELSE IF risk_score > 40 THEN
14     status = "WARNING"; red = OFF; green = OFF
15 ELSE
16     status = "NORMAL"; red = OFF; green = ON
17 END IF
18 Publish V2/V3/V5/V6 to Blynk via HTTPS GET
19 ESP32 receives V3 and applies GPIO25/26 LED states
20 Append timestamp, passengers, temp, weather, risk,
    status to bus_stop_log.csv locally
21 Continue next cycle; handle API/file errors

```

Figure 4.8: Risk Classification Pseudocode

Figure 4.8: Risk evaluation, LED return path and Telegram cooldown

The backend cannot directly toggle ESP32 GPIO across the network. It publishes V3 through Blynk, and the ESP32 callback converts the received risk into GPIO25/26 states. V6 carries the displayed status/routing message. Figure 4.8 makes that return path explicit and resets the non-selected LED so a previous state cannot remain asserted.

The pseudocode permits the first CRITICAL alert immediately and then requires at least 60 seconds after the last successful alert. Logging is a separate local operation at the end of each completed cycle. HTTP failures and timeouts must be handled so one service failure does not terminate the monitoring loop. A nominal cycle interval is not a guaranteed real-time deadline.

Boundary checks should include scores of 40.0, 40.5, 75.0 and just above 75; counts of 0 and below 30 for manual subtraction; invalid DHT22 readings; failed weather retrieval; repeated blockage; and repeated CRITICAL cycles inside the alert cooldown. These are acceptance checks, not fabricated observations.

4.9 SAFETY FEATURES IMPLEMENTED

The prototype combines low-voltage hardware, visual rather than audible escalation, private credentials and controlled alerting. The implementation status below distinguishes the specified safeguards from features that have not been verified or completed.

Planned feature	Implemented?	Implementation notes
Low-voltage peripheral supply	Yes	5 V USB enters the ESP32; sensors use its regulated 3.3 V rail. This does not remove short-circuit or moisture risks.
LED current limiting	Yes	Separate 220-ohm resistor in each GPIO25/GPIO26 LED branch.
Audible alarm prevention	Yes	Buzzer retained through BC547 but disabled in firmware.
Critical-alert cooldown	Yes	Telegram eligibility is limited by a 60 s cooldown.
HTTPS for backend APIs	Yes, specified	Blynk REST, OpenWeatherMap and Telegram requests use HTTPS.
ESP32-to-Blynk TLS	Not verified	BlynkSimpleEsp32.h alone does not demonstrate encrypted device transport.
Credential protection	Specified	Keep tokens, API keys and Wi-Fi credentials out of public documents, repositories and diagnostic logs.
Privacy-preserving sensing	Yes	Distance/count and temperature only; no faces or device identifiers are required.
Invalid-data handling	Yes	Reject DHT22 NaN; backend defaults to 28.0 C and Clear when unavailable. Defaults may underestimate risk.
Protective enclosure	Pending	Exposed breadboard illustrations do not establish weatherproofing or outdoor certification.

The development system is not a certified public-safety device. Operators must not treat fallback values or stale displays as proof that conditions are safe. Public installation requires protected connections, secure mounting, moisture protection and a verified recovery behaviour after power or network loss.

4.10 PROTOTYPE DEVELOPMENT

The prototype integrates the ESP32, VL53L0X, DHT22, red/green indicators and local operator button on a breadboard. USB power supplies the ESP32, which distributes regulated 3.3 V to the sensors. The disabled buzzer provision remains part of the design. The local Windows computer runs `brain.py`; the model does not execute on the ESP32 or inside Blynk Cloud.

Integration separated the ownership of cloud values: the ESP32 writes V1, V4 and V8, while Python writes V2, V3, V5 and V6. V7 carries the operator command; GPIO27 is its local counterpart. This avoids competing writers for the same measurement. A changed count is evaluated on a subsequent backend cycle, then V3 returns the risk to the ESP32 for LED control.

The supplied project description reports sensor-to-cloud operation, backend processing and critical alert delivery. The current documentation represents a functional development prototype, not independently verified field performance or a completed outdoor enclosure. The generated views in Section 4.11 illustrate assembly and demonstration intent only.

Prototype Specifications

Specification	Details
Power supply	5 V USB to ESP32; 3.3 V peripheral rail
Main controller	ESP32 DoIT DevKit V1 with Wi-Fi
Sensors	VL53L0X distance; DHT22 temperature
Outputs	Red/green LEDs; Blynk display; Telegram alerts
Manual controls	Blynk V7 and local GPIO27 button; subtract 30, floor at zero
Audible output	Buzzer/BC547 provision; disabled
Sensor timing	VL53L0X 150 ms; DHT22 3 s
Backend cycle	Nominal 5 s, subject to processing/network time
Machine learning	RandomForestRegressor; 100 trees; <code>random_state=42</code>
Persistent data	<code>bus_stop_log.csv</code> on the backend computer
Stop / route	BEST-DIN-001, Dindoshi Bus Depot, BEST Route 326

Current status

Functional development prototype as reported;
quantitative validation not supplied

Practical limitations include single-sensor crossing ambiguity, manual estimation of boarding passengers, dependency on Wi-Fi and the backend computer, external API quotas and conservative interpretation of fallback data. These constraints define the scope of the prototype and guide further validation.

4.11 PROTOTYPE PHOTOS AND DEMONSTRATION

This section retains the reference chapter's sequence of top view, side view, component close-up, dashboard and demonstration.

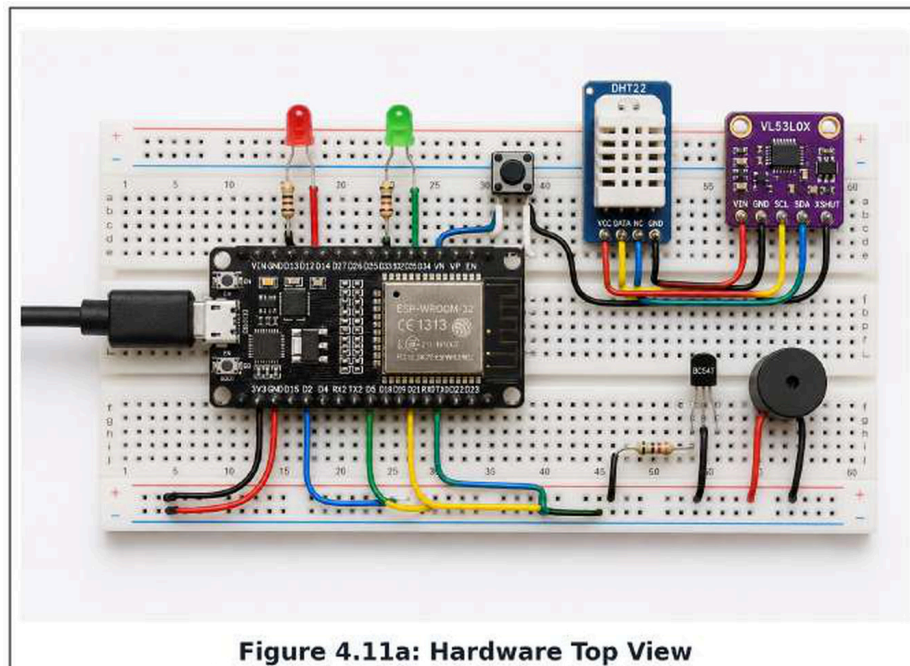


Figure 4.11a: Top view of the breadboard prototype

The top view illustrates the main breadboard assembly, including the controller, distance and temperature modules, indicators, operator button and disabled buzzer provision. Physical pin connections are defined in Section 4.3.

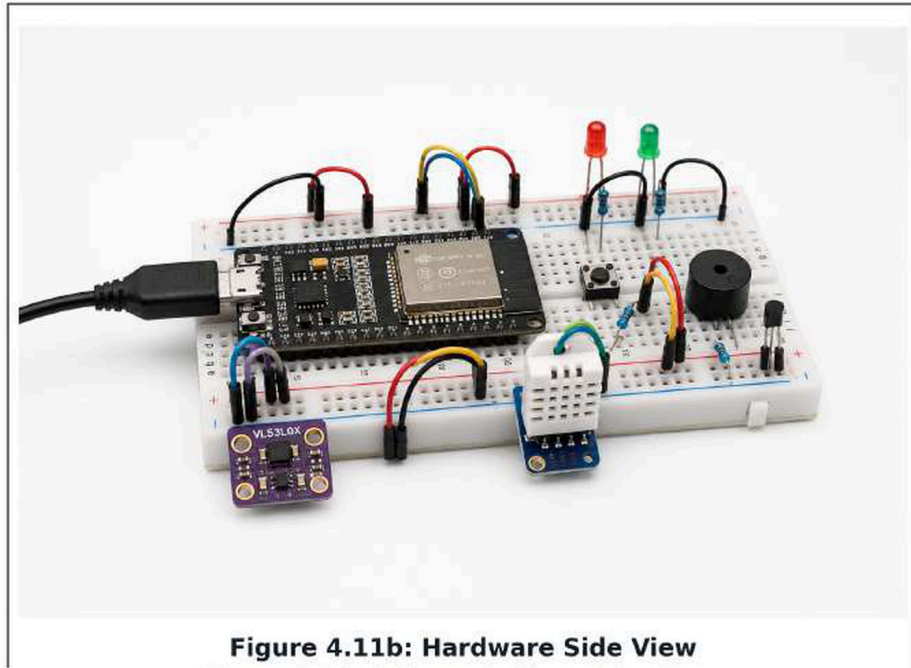


Figure 4.11b: Side view of component mounting

The side view illustrates component mounting and jumper-wire access. It does not establish a final enclosure, wiring continuity or tested mechanical robustness.



Figure 4.11c: VL53L0X sensor close-up

The sensor close-up identifies the VL53L0X optical module used for distance measurements. Two consecutive readings below 400 mm are required before a crossing increments V1; a stationary obstruction is not repeatedly counted.

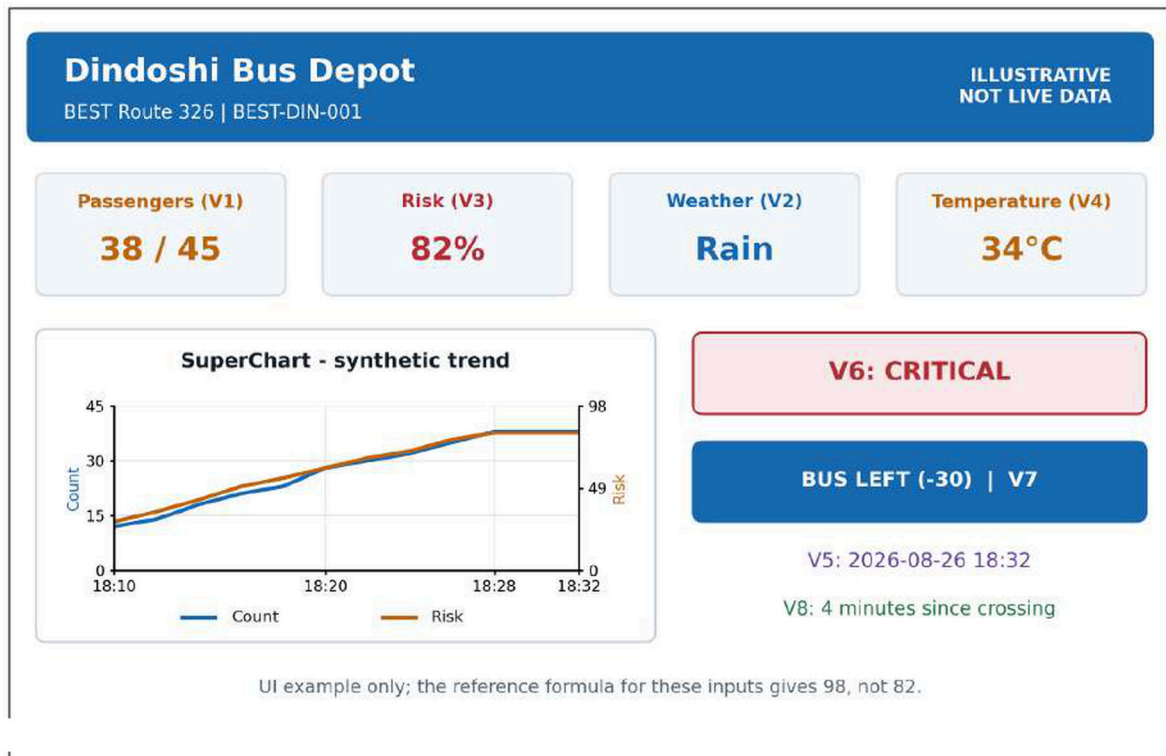


Figure 4.11d: Dashboard mockup repeating the Section 4.6 example

The dashboard mockup repeats the V1-V8 example from Section 4.6. The risk/status pair 82% and CRITICAL is internally consistent, while the full set of displayed inputs is illustrative rather than a verified model output. The synthetic chart uses labelled scales and ends at 38 passengers and 82 risk points.

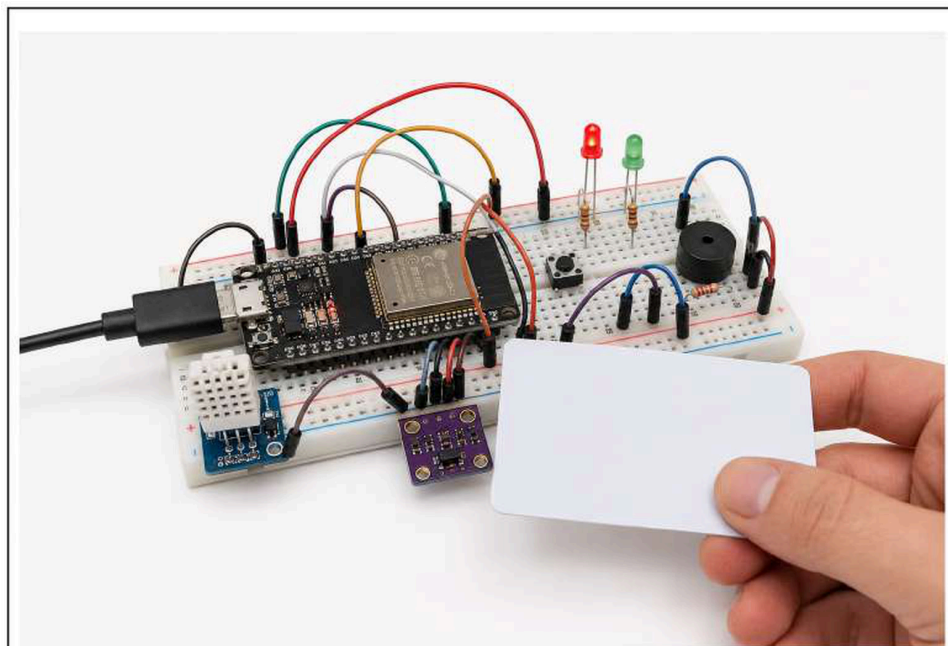


Figure 4.11e: System in Action - CRITICAL State

Figure 4.11e: Demonstration of the accumulated CRITICAL LED state**Demonstration Scenario**

An object is moved through the VL53L0X detection region and then removed. When two consecutive usable readings are below 400 mm, the ESP32 increments the passenger count and updates V1; removal re-arms the detector. The backend retrieves the updated count, combines it with temperature, weather and local hour, and predicts risk. It publishes the result on V3 and the associated display values on V2, V5 and V6.

For a predicted value above 75, the ESP32 illuminates the red LED and turns the green LED off. Telegram is notified only when the 60-second cooldown permits, and the completed cycle is appended to the CSV log. The red-lit illustration depicts that accumulated CRITICAL state; it does not imply that one object alone caused a critical crowd.

The operator can press either V7 or the local GPIO27 button to subtract 30 passengers without going below zero. The backend then reevaluates the reduced count. A complete evidence record would pair real top/side photographs with the Blynk screen, Serial Monitor output, a CSV row and the actual alert timestamp from the same test.

Chapter Summary

Chapter 4 documents the development environment, phased build process, hardware assembly, firmware modules, cloud exchange, dashboard, regression-based risk logic and safety provisions. The chapter uses consistent sensor pins, virtual-pin ownership, timing and thresholds across the text and diagrams. Quantitative performance claims remain dependent on the actual testing evidence, while the supplied illustrations communicate the prototype's structure and intended operation.